Høyskolen Kristiania

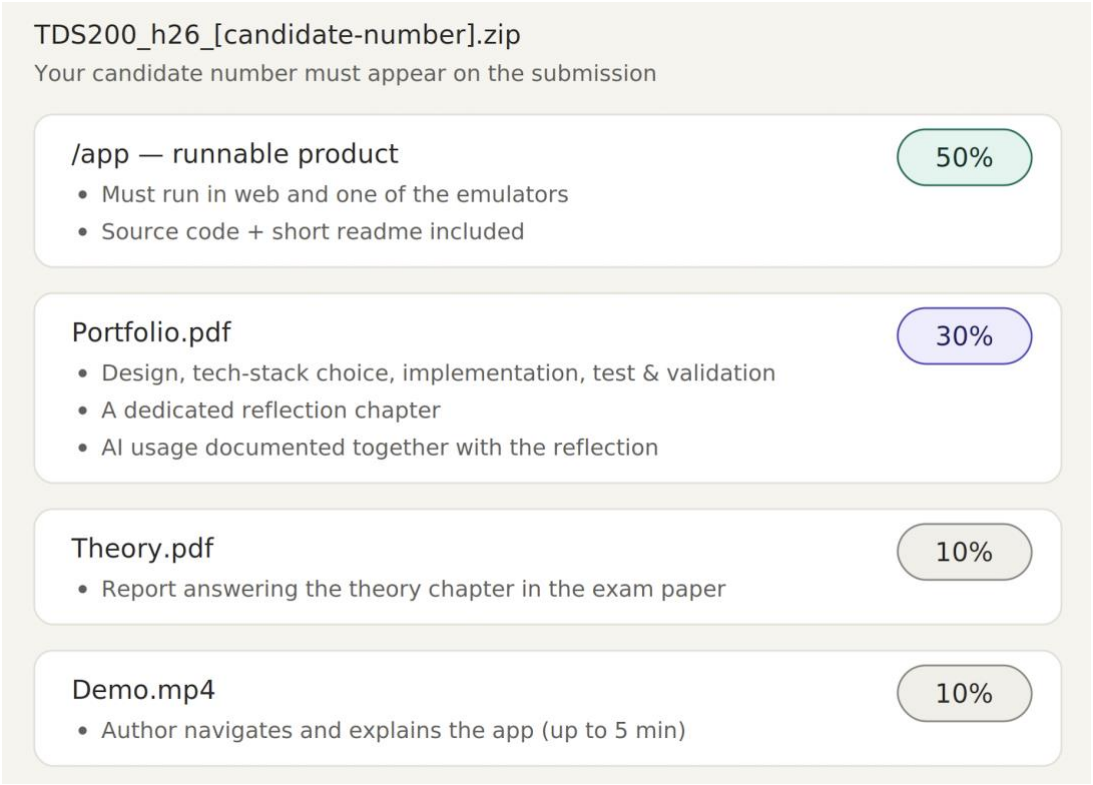
Course code: TDS200
Subject name: Kryssplattform
Exam format: Written home exam individually or in group up to 3 members
Delivery/submission date:
File format: .zip

Remember to state your candidate number on your delivery folder.

Høyskolen Kristiania

EXAM
TDS200 Kryssplattform
Autumn 2026

Your submission should follow the below structure, and your performance will be evaluated based on both technical and theoretical competence.



Important Information

- This home exam can be solved and delivered individually or in group up to 3 members. It is expected that a group submission shall accomplish more, i.e., in terms of code volume and complexity etc than individual submission.
- Assessment guide for internal and external assessors, also as a reference for students, can be found at the separate sensorveiledning document.
- Read through the entire exam text before starting.
- Always follow good referencing practice. When using quotes, code snippets, statistics, images, figures, or similar elements that you have not created yourself, it is essential to provide proper citations. Follow the referencing guidelines established by Høyskolen Kristiania.

Remember to state your candidate number on your delivery folder.

Høyskolen Kristiania

- Plagiarism must not occur. It is not allowed to copy code without providing a proper reference. Using TDS200 course lecture code is not regarded as plagiarism.
- Using component library and npm modules is not considered plagiarism, and it is encouraged to demonstrate your competence utilizing npm modules and third-party libraries. Remember to provide appropriate references too.

Høyskolen Kristiania

Task 1 – ChallengeResolver App (weighting: 50%)

You should design and implement a cross-platform mobile application called **ChallengeResolver,** where users can discover and complete location-based street challenges.

A challenge may ask the user to go to a specific place and verify something in the environment, for example,

- Find a statue or landmark
- Take a photo of a specific object
- Identify information shown on a sign

The candidate shall use **React Native with Expo** for frontend development (in alignment with the technologies covered in the course), and **Firebase** as Backend as a Service.

Key evaluation criteria:

- The solution must be built and run without errors and can be run with Web browsers, as well as Android emulator and/or iOS simulator.
- Comment the code to illustrate your understanding of code and explain complex logic when necessary.
- The code should adhere to best practices to enhance code quality, including proper naming conventions, code structures and emphasizing code reusability.
- Basic error handling is expected.
- Ensure the app is user-friendly with all functionalities working as expected.
- Any external libraries, modules, or packages required for the project should be integrated appropriately.

Creation and delivery of exam project:

- Create your own Firebase and there must be some data available at Firebase so that the exam sensor can see that the solution works.
- Please also attach a readme.md file to describe what functionality you have implemented in the app and whether you have run it on Android emulator, iOS simulator or both.
- Remove the node_modules folder before uploading to WISEFLOW!
- Make sure the package.json file contains all npm dependencies, expecting the exam sensor to install npm packages with the command “npm install”
- The project folder you delivered shall be named as “TDS200_h26_your candidate number.zip”
- Your project delivery must include a .env file containing the necessary Firebase configuration keys and Google API keys if you use those services. These keys are

project specific. After the exam has been ass, you may safely delete the associated Google Cloud and Firebase projects you made for the exam.

- You are expected to provide at least one test user account for email/password authentication (documented in readme.md) and prepare a few sample Challenge cases so that the examiner can log in with the test user account and verify your application´s functionality.

Primary requirements for implementation

The following requirements represent the minimum expected implementation. The numbering does not indicate priority**:**

1. **Challenges Overview**: Display available challenges together with the authenticated user´s progress status:
    - Not started
    - In progress
    - Completed

    The user must be able to open a challenge details screen by clicking a challenge item or create a new challenge.

2. **Challenge Details:** Each challenge must contain at least: title, description, question, target location (latitude, longitude), valid completion radius.

3. **Resolve Challenge**: A screen that loads the selected challenge and displays its question and completion requirements. User must fulfil all the following conditions to complete the challenge:
    - Verify current GPS Location.
    - Capture or select a photo.
    - Answer a question

    The challenge can only be marked as **Completed** when all required conditions are fulfilled.
    Captured or selected images used as evidence to resolve the challenge must be uploaded and stored appropriately using Firebase Storage.

4. **Location Verification and Mock Location**: Each challenge must define
    - A target location (latitude and longitude)
    - A valid completion radius

    The application must provide a **Mock Location mode** that allows the examiner to simulate location verification. This can be implemented by manually entering latitude and longitude coordinates or selecting predefined test locations (e.g., within radius,

outside radius) from a dropdown menu. The simulated location must be used to determine whether the challenge´s location verification requirement is satisfied.

5. **User Authentication and Progress**: Users must register and log in using email/password through Firebase Authentication to access the application. Challenges are shared among users, but each authenticated user´s challenge resolving progress must be tracked and updated independently.

Extended functionality

The five primary requirements mentioned above present the minimum requirement for the assignment. For those aiming for a higher grade, more functionalities and complexity in the implementation is expected. Please check below list to see examples of functionalities that can contribute to a higher grade. Possible extensions include and the difficulty level are marked as below:

Basic-complexity extensions:

- **Filtering and Sorting**: Filter by status, category, difficulty, or location.
- **Enhanced Error Handling**: Provide comprehensive error handling, including meaningful error messages, loading states, unavailable services, and denied permissions.
- **Interactive Communication**: Users can rate, like, comment on, or other interactive communications.
- **Multiple Photo Evidence**: Allow users to capture or upload several images and manage them such as rotating or cropping them before upload.

Medium-complexity extensions:

- **Challenge Editing and Management**: Allow users to edit and delete challenges they have created.
- **Progress and Scoring**: Award points for completed challenges and display overall progress, levels, badges, or a leaderboard.
- **Security and Validation**: Implement client-side authorization, data validation, and Firebase security rules to prevent unauthorized access. In this case, a test user account must be provided to allow the examiner to validate the implementation.

Advanced-complexity extensions:

- **Enhanced Authentication**: The app supports multiple login options, including email/password and social logins such as Google Sign-In.

- **Interactive Map**: Integrate an interactive map to display challenge locations and/or user´s real or simulated locations. The map may be implemented as a separate tab showing all challenge locations and/or integrated into individual challenge details screens. Additional map-related features may be implemented as desired.

- **Role-based Access Control.** Introduce roles such as Challenge organizers and Challenge participants. For example, organizers may create, edit, and delete challenges and manage participants, while participants can view start, and resolve challenges.

Simply adding many unrelated features does not automatically result in a higher grade. Extended functionality should be technically meaningful and well-integrated into the application.

Task 2 – Portfolio (weighting: 30%)

You shall submit a portfolio documenting the design, development, testing, and evaluation of the application developed in Task 1- ChallengeResolver App.

The portfolio should demonstrate both technical understanding and critical reflection on the development process.

1. Design and technology choices

Describe the overall design of the application and explain the main technical choice you have made. This should include, where relevant:

- Application structure and navigation design
- Component structure
- **Data model and Firebase structure (must-to-have)**
- Choice of libraries, frameworks and APIs.
- Integration of native device functionality
- Important UI/UX design decisions

Technology choices should be justified rather than only listed.

2. Implementation

Describe and demonstrate the implementation of the most important parts of the application. Focus on technically significant functionality rather than documenting every line of code. Relevant examples my include:

- Firebase integration
- Location verification and Mock Location mode
- Map integration (if implemented)
- Reusable components

- Asynchronous operations and error handling

3. Testing and validation

Describe how the application was tested and validated, including both successful and unsuccessful scenarios. Examples may include:

- Location inside the challenge radius
- Location outside the challenge radius
- Valid and invalid user input
- Permission granted and denied
- Network or Firebase errors

4. Critical Reflection

The portfolio must contain a dedicated reflection chapter. This chapter should critically reflect on the development process rather than only describing what was done. You should discuss topic as:

- What worked well
- What did not work as expected
- Important technical or design decisions
- Compromises made
- What you learned during the project
- What you would redesign or improve if you continued the project

5. Generative AI usage

The portfolio must include a dedicated section documenting the use of generative AI tools during the exam assignment. If AI tools are used, document

- Which AI tool(s) were used
- Examples of significant prompts or interactions
- How AI-generated suggestions or code were evaluated
- How you tested and verified that the final solution was correct
- Which parts of the submitted work were completed independently

If no generative AI tools were used, this should be stated explicitly.

Task 3 – Theory (weighting: 10%)

Question 1: Cross-platform Development, Expo Go and Native Features (5%)

Explain how React Native supports cross-platform mobile development and how Expo Go enables access to native features, such as the camera or location services. Compare the

React Native approach with other cross-platform development approaches and with native app development, discussing their advantages and limitations.

Question 2: Application Architecture (5%)

Explain why separating UI components, application logic, and backend access is important in React Native development. Use examples from your ChallengeResolver App to demonstrate how this separation improves maintainability, reuse and testability.

Task 4 – The Video (weighting: 10%)

Make a short video called demo.mp4 (screen recording) of the app showing the functions you have implemented. Limit the video to no more than 5 minutes and ensure it highlights any advanced features that may affect the grade.